

SQL Medium Level Question and Answer:

12 January 2026 20:26

THEORY QUESTIONS:

Question 1] What is the difference between INNER JOIN and OUTER JOIN?

Answer]

- **INNER JOIN** returns only matching records from both tables.
- **OUTER JOIN** returns matching records **plus non-matching records**.
 - LEFT JOIN → all records from left table
 - RIGHT JOIN → all records from right table
 - FULL JOIN → all records from both tables

Question 2] What is a correlated subquery?

Answer] A **correlated subquery** depends on the outer query and executes **once per row** of the outer query.
Slower than normal subqueries but useful for row-wise comparisons.

Question 3] What are window functions?

Answer] Window functions perform calculations across a **set of rows related to the current row**, without collapsing rows.

Examples:

- ROW_NUMBER()
- RANK()
- DENSE_RANK()
- SUM() OVER()

Question 4] Difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

Answer]

Function	Duplicate Rank	Gaps
ROW_NUMBER	No Duplicate ranks	No gaps
RANK	Contains duplicate ranks	also contains gaps
DENSE_RANK	Contains duplicate ranks	No gaps

Question 5] What is denormalization?

Answers] Denormalization is the process of **adding redundancy** to improve read performance, often used in analytics systems.

Question 6] What is a composite key?

Answer] A **composite key** is a primary key made up of **two or more columns** to uniquely identify a record.

Question 7] What is indexing overhead?

Answer] Indexes improve SELECT performance but:

- Increase storage usage
- Slow down INSERT, UPDATE, DELETE operations

Question 8] What is the difference between UNION and JOIN?

Answer]

- **UNION** combines rows from multiple SELECT statements.
- **JOIN** combines columns from multiple tables based on a condition.

Question 9] What are CTEs (Common Table Expressions)?

Answer] CTEs are temporary named result sets used to simplify complex queries.

Syntax:

```
WITH cte_name AS (  
  SELECT ...  
)  
SELECT * FROM cte_name;
```

Question 10] What is query optimization?

Answer] Improving query performance by:

- Using indexes
- Avoiding SELECT *
- Filtering early
- Using joins instead of subqueries

PRACTICAL QUESTIONS:

Question 11] Find the **third highest salary** from an employee table.

Answer]

```
SELECT DISTINCT salary  
FROM employee  
ORDER BY salary DESC  
LIMIT 1 OFFSET 2;
```

Question 12] Get employees earning **above department average salary**.

Answer]

```
SELECT e.name, e.salary
FROM employee e
JOIN (
    SELECT department, AVG(salary) avg_sal
    FROM employee
    GROUP BY department
) d
ON e.department = d.department
WHERE e.salary > d.avg_sal;
```

Question 13] Find duplicate records with full details.

Answer]

```
SELECT *
FROM employee
WHERE email IN (
    SELECT email
    FROM employee
    GROUP BY email
    HAVING COUNT(*) > 1
);
```

Question 14] Display the **highest salary in each department**.

Answer]

```
SELECT department, MAX(salary)
FROM employee
GROUP BY department;
```

Question 15] Find employees who joined **before their manager**.

Answer]

```
SELECT e.name
FROM employee e
JOIN employee m
ON e.manager_id = m.emp_id
WHERE e.joining_date < m.joining_date;
```

Question 16] Fetch **top 2 salaries per department**.

Answer]

```
SELECT *
FROM (
    SELECT name, department, salary,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) rnk
    FROM employee
) t
WHERE rnk <= 2;
```

Question 17] Find departments with **more than 10 employees**.

Answer]

```
SELECT department
FROM employee
GROUP BY department
HAVING COUNT(*) > 10;
```

Question 18] Get employees whose names start and end with the same letter.

Answer]

```
SELECT name
FROM employee
WHERE LEFT(name,1) = RIGHT(name,1);
```

Question 19] Calculate **running total of salaries**.

Answer]

```
SELECT name, salary,
SUM(salary) OVER (ORDER BY salary) AS running_total
FROM employee;
```